

# LMDA: Lightweight Multiauthority CP-ABE Scheme Based on Dynamic Attributes for System Wide Information Management

Lizhe Zhang<sup>1</sup>, Yiao Ma<sup>1</sup>, Zhijun Wu<sup>1</sup>, Kenian Wang<sup>1</sup>, and Ruiqi Li

**Abstract**—The system wide information management (SWIM) system serves as an advanced platform for managing and sharing diverse aviation data, such as aeronautical, flight, and meteorological information. With the exponential growth of aviation data, ensuring data security and privacy within SWIM has become increasingly important. Ciphertext-policy attribute-based encryption (CP-ABE) is widely regarded as an effective cryptographic technique for ensuring secure and private data access. However, traditional CP-ABE schemes rely on computationally expensive bilinear pairings, which are unsuitable for resource-constrained aviation Internet of Things (IoT) application scenarios in SWIM, such as Aircraft Access to SWIM (AAtS). To address this limitation, we propose a lightweight multiauthority CP-ABE scheme based on dynamic attributes. This scheme replaces bilinear pairing operations with efficient scalar multiplication on elliptic curve points and outsources most decryption operations to outsourcing decryption nodes, thereby significantly reducing computational overhead. In addition, this scheme introduces a dynamic attribute mechanism to support fine-grained access control, and outsources data verification and attribute revocation to the blockchain to ensure data integrity. Security analysis shows that our scheme achieves indistinguishability under adaptive chosen-ciphertext attacks (IND-CCAs) and is resistant to collusion attacks. Experimental evaluations further demonstrate that our scheme significantly enhances computational and communication efficiency.

**Index Terms**—Ciphertext-policy attribute-based encryption (CP-ABE), dynamic attributes, elliptic curve cryptography (ECC), multiauthority, system wide information management (SWIM).

## NOMENCLATURE

|                   |                          |
|-------------------|--------------------------|
| $GID$             | Global identification.   |
| $A$               | User attribute set.      |
| $PK_{PC}$         | Public key of PC.        |
| $SK_{PC}$         | Private key of PC.       |
| $(\Lambda, \rho)$ | Access structure.        |
| $UA$              | Universal attribute set. |

|             |                                                             |
|-------------|-------------------------------------------------------------|
| $UK$        | Temporary universal attribute key.                          |
| $UK'$       | Universal attribute key.                                    |
| $K$         | Symmetric encryption key.                                   |
| $CT_H$      | Signature for integrity verification.                       |
| $t$         | Hourly epoch number.                                        |
| $AAD$       | Additional authenticated data.                              |
| $PKA$       | Attribute public key.                                       |
| $MK$        | Master secret key.                                          |
| $\Lambda_x$ | $x$ th row of matrix $\Lambda$ .                            |
| $AK$        | Temporary attribute key.                                    |
| $AK'$       | Attribute key.                                              |
| $MD$        | Model attribute set.                                        |
| $MDK$       | Temporary model attribute key.                              |
| $MDK'$      | Model attribute key.                                        |
| $CT_K$      | Ciphertext generated by encrypting data using the key $K$ . |
| $CT$        | Complete ciphertext structure.                              |
| $nonce$     | Per-encryption 96-bit AEAD nonce.                           |
| $RL_{ver}$  | Revocation list version number.                             |

## I. INTRODUCTION

AS A large-scale aviation information exchange platform, the system wide information management (SWIM) has become an essential component of air traffic management (ATM) systems [1]. The International Civil Aviation Organization (ICAO) has recognized SWIM as a core technology for global ATM interoperability and information sharing. Accordingly, ICAO has been actively promoting the development and implementation of its information service standards [2]. SWIM adopts a service-oriented distributed architecture [3], which provides users with unified information-sharing services through the ICAO meteorological information exchange model (IWXXM), the flight information exchange model (FIXM), and the aeronautical information exchange model (AIXM). It supports the standardization of communication for navigation, surveillance, meteorological, and aeronautical data, and offers flexible data publishing and subscription mechanisms to meet the demands of civil aviation.

Fig. 1 illustrates the SWIM system architecture, which is a service-oriented information-sharing platform. Through the air traffic control (ATC) big data center, it integrates various levels of ATC units, such as tower, area, approach, and flow management, as well as ATC functional departments, including aviation meteorology, navigational intelligence, operations management, and technical support. It also establishes data connections with ground operation entities such as airports and

Received 28 December 2025; revised 26 February 2026; accepted 20 March 2026. Date of publication 27 March 2026; date of current version 9 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62571531 and in part by the Education Commission Scientific Research Project of Tianjin under Grant 2022KJ081. (Corresponding author: Zhijun Wu.)

Lizhe Zhang, Yiao Ma, Zhijun Wu, and Ruiqi Li are with the School of Safety Science and Engineering, Civil Aviation University of China, Tianjin 300300, China (e-mail: lzzhang@cauc.edu.cn; 2023092118@cauc.edu.cn; zjwu@cauc.edu.cn; rq\_li@cauc.edu.cn).

Kenian Wang is with the Institute of Scientific Innovation, Civil Aviation University of China, Tianjin 300300, China (e-mail: knwang@cauc.edu.cn).

Digital Object Identifier 10.1109/IJOT.2026.3678416

2327-4662 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Lovely Professional University - Phagwara. Downloaded on August 20, 2026 at 10:56:11 UTC from IEEE Xplore. Restrictions apply.

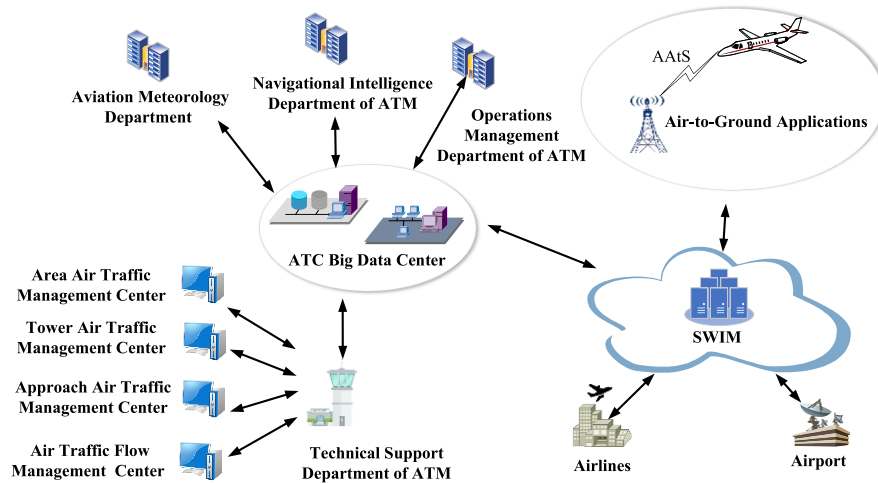


Fig. 1. SWIM system architecture.

airlines, to enable bidirectional sharing of key data, including flight plans, flight status, and ground operations.

In addition to supporting data sharing for ground-based systems, SWIM also provides real-time data-sharing services for in-flight aircraft through the aircraft access to SWIM (AAtS) mechanism. This enables the aircraft to dynamically obtain ground information such as flight operation status, weather updates, and airspace usage, thereby enhancing situational awareness and supporting autonomous decision-making in flight. As a key component of the aviation Internet of Things (IoT), AAtS supports both low-latency and highly reliable data communication. Consequently, it plays an essential role in the aviation IoT communication architecture. The introduction of AAtS effectively extends the application scope of SWIM to integrated air-ground collaborative operations, significantly improving the transparency and responsiveness of air traffic operations. However, as a typical air-ground data exchange mechanism, AAtS is also exposed to various security threats, including unauthorized access and modification, illicit disclosure, denial of access, and other security threats [4]. Deploying effective security mechanisms is essential to prevent data leakage and protect user privacy, thereby mitigating these threats. Among these, access control is considered a key solution. It is noteworthy that AAtS relies on air-ground communication links with limited bandwidth. Therefore, when designing data access control mechanisms, it is critical to balance security with computational efficiency, which highlights the need for lightweight access control solutions.

#### A. Contribution

We propose a lightweight multiauthority ciphertext-policy attribute-based encryption (CP-ABE) scheme for SWIM based on dynamic attributes, which refer to status-varying and context-dependent attributes that can change with SWIM operational conditions. The main contributions of our work are as follows.

- 1) We adopt lightweight scalar multiplication in place of traditional bilinear pairing to reduce internal

computational complexity. Furthermore, most decryption tasks are outsourced to the outsourcing decryption nodes, effectively lowering the computational load.

- 2) We improve the CP-ABE scheme by incorporating three SWIM information exchange models (AIXM, FIXM, and IWXXM) as dynamic attributes to achieve fine-grained access control. These dynamic attributes introduce model-specific access constraints that ensure clients can only interact with ciphertexts associated with their authorized models.
- 3) We outsource data verification and attribute revocation to the blockchain. The blockchain records the public key used for verification, allowing clients to check the publisher's identity and the integrity of the data. At the same time, the attribute authority (AA) updates the revocation list to prevent the distribution of ciphertexts to revoked clients.
- 4) We perform a comprehensive security analysis and experimental evaluation of our scheme. The results show that our scheme achieves indistinguishability under adaptive chosen-ciphertext attacks (IND-CCA) and resists collusion attacks, while also achieving forward security. Our scheme demonstrates higher efficiency compared with most of the existing schemes in terms of computational and communication overhead.

#### B. Organization

The rest of this article is structured as follows. Section II presents related research on CP-ABE and access control in SWIM. Section III discusses the scheme's preliminary concepts. Section IV provides an overview of the scheme. Section V provides the details of the scheme. Section VI analyzes the scheme's security. Section VII evaluates the scheme's performance. Section VIII concludes this article.

## II. RELATED WORK

Bethencourt et al. [5] initially proposed the concept of CP-ABE, which has since received significant attention from

researchers. Depending on the application scenarios, CP-ABE represents access policies using different access structures, including logical AND/OR gates, threshold-based access trees, and linear secret sharing schemes (LSSSs). User attributes and their corresponding cryptographic keys are managed through either a single-authority architecture [6] or a multiauthority architecture [7]. Several variants of CP-ABE have been developed to address diverse functional and security requirements. These include hierarchical CP-ABE [8], outsourced CP-ABE [9], [10], traceable CP-ABE [11], revocable CP-ABE [12], [13], and policy-hiding CP-ABE [14], [15]. CP-ABE ensures that only users possessing the required attribute set can access the encrypted data. In addition, it simplifies key management and enhances both system security and privacy protection, making it applicable to a wide range of scenarios with high security demands.

Many current CP-ABE solutions rely on bilinear pairings, which impose heavy computational demands and are impractical for devices with limited resources. To address this limitation, researchers have proposed replacing bilinear pairings with elliptic curve scalar multiplication to improve efficiency while preserving data confidentiality. Ding et al. [16] introduced the first lightweight pairing-free CP-ABE scheme, which allows for the direct revocation of users without requiring updates to other keys. However, the scheme relies on a centralized trust authority, making it vulnerable to single points of failure. Therefore, Ding et al. [17] further combined blockchain technology with CP-ABE, thereby mitigating the risks of data tampering and single-point failure. Nevertheless, the scheme still has the key-escrow problem. Sowjanya et al. [18] proposed a key-escrow-free hierarchical CP-ABE scheme to overcome this issue, reducing the burden on centralized authorities. However, its applicability is limited to single-authority scenarios, and it supports only AND-gate access structures, restricting its policy expressiveness. In response to the increasing demand for multiauthority architectures, Cheng et al. [19] developed a CP-ABE scheme for the electric IoT that integrates temporal and spatial attribute constraints to support fine-grained access control. However, the proposed scheme cannot verify data integrity. Sammy and Vigila [20] introduced a CP-ABE framework enhanced with blockchain that supports collaborative attribute management among multiple authorities. Das and Namasudra [21] introduced outsourced decryption in a healthcare IoT setting to offload computational overhead from user devices and support multiauthority key generation. Xie et al. [22] developed a CP-ABE scheme that integrates blockchain and IPFS, utilizing smart contracts to achieve verifiable access control. Although these schemes enhance decentralization and verifiability to some extent, most of them lack efficient mechanisms for attribute revocation. Das and Namasudra [23] addressed this by proposing a key-update-based attribute revocation scheme suitable for multiauthority environments. This scheme dynamically updates the attribute keys of users to revoke access, but it requires re-encryption of ciphertexts and re-updating the key, which is time-consuming and lacks verification capabilities. While these elliptic curve cryptography (ECC)-based schemes improve computational performance, they are still limited in their ability to address

the fine-grained access control requirements of the SWIM data exchange model, particularly in terms of balancing efficiency and verifiability in attribute revocation.

Research on access control in the SWIM environment has primarily focused on two technical approaches: attribute-based access control (ABAC), emphasizing dynamic role-attribute authorization frameworks, and CP-ABE, leveraging fine-grained cryptographic policies for protecting data. Wu et al. [24] developed a CP-ABE scheme with attribute revocation tailored for SWIM. The scheme integrates proxy re-encryption and a key-encryption key tree structure. Subsequently, Wu et al. [25] proposed a verifiable CP-ABE scheme for the SWIM cloud-based management center by introducing a secret sharing mechanism capable of identifying malicious users. However, this scheme lacks sufficient expressiveness when handling complex access policies. Building upon this, research attention has shifted toward attribute correlation and model optimization. Based on the ABAC, Zhang et al. [26] combined monotonic span programs with LSSS to restructure the traditional eXtensible Access Control Markup Language model. They further proposed a dynamic authorization algorithm based on attribute correlation, which significantly enhances the expressiveness of access policies. Nevertheless, this scheme fails to ensure the integrity of access data. With the growing adoption of blockchain technology in ATM systems, blockchain-based approaches have also been introduced into security research for SWIM. Wang et al. [27] designed a cloud-chain-fusion CP-ABE scheme to support multiauthority outsourced computation, verification, and auditability. Lu et al. [28] designed a blockchain-based access control framework for ATM, which is well aligned with the operational characteristics of SWIM as a core component of ATM.

Existing schemes have high computational overhead, and many of them lack support for multiauthority architectures, attribute revocation, and verifiable decryption, which makes them unsuitable for resource-constrained SWIM applications. To address these limitations, we propose a lightweight multiauthority CP-ABE scheme based on dynamic attributes (LMDA). Table I summarizes the evaluation of our scheme in comparison with other existing SWIM-related access control schemes. Compared with existing schemes, our scheme offers significant advantages in computational efficiency, attribute revocation support, and verifiability of decryption results. It effectively addresses the challenges of balancing security, efficiency, and fine-grained control in SWIM with constrained computing resources, offering enhanced security and practical applicability.

### III. PRELIMINARIES

In this section, we describe the preliminaries of our scheme, including ECC, access structure and attribute, LSSS, and Decisional Diffie-Hellman (DDH).

#### A. Elliptic Curve Cryptography

The concept of using elliptic curves in cryptography was initially and independently introduced by Miller [29] and Koblitz [30]. These curves possess a fundamental mathematical structure defined over a finite field of prime numbers. The

TABLE I  
COMPARISON OF SWIM-RELATED ACCESS CONTROL SCHEMES

| Scheme | Research Method | Multiauthority | Attribute-revocable | Verifiable | Fine-grained | Lightweight |
|--------|-----------------|----------------|---------------------|------------|--------------|-------------|
| [24]   | CP-ABE          | ×              | √                   | ×          | √            | ×           |
| [25]   | CP-ABE          | √              | ×                   | √          | √            | ×           |
| [26]   | ABAC            | ×              | ×                   | ×          | √            | ×           |
| [27]   | CP-ABE          | √              | √                   | √          | √            | ×           |
| [28]   | ABAC            | ×              | √                   | √          | √            | ×           |
| ours   | CP-ABE          | √              | √                   | √          | √            | √           |

security of ECC relies on the computational difficulty of the elliptic curve discrete logarithm problem (ECDLP) [31].

The domain parameters  $(q, a, b, G, r)$  characterize an elliptic curve  $E$  defined over a prime finite field.  $q$  represents the order of the finite field  $F_q$ ,  $G$  is a generator,  $r$  represents the order of the subgroup generated by the base point  $G$  on the elliptic curve  $E$ . An elliptic curve  $E$  over  $F_q$  is represented as

$$E: y^2 = x^3 + ax + b, \quad \text{where } 4a^3 + 27b^2 \neq 0. \quad (1)$$

ECC achieves comparable security with smaller key sizes than RSA, primarily because solving ECDLP is computationally more challenging than factoring large integers. In ECC, given a generator  $G$  of a group with prime order, computing the scalar  $k$  for a given point  $Q = k \cdot G$  is a challenging task that remains resistant to polynomial-time solutions.

Consider two clients,  $C_a$  and  $C_b$ , seeking to communicate securely. The two clients each select the private key  $MK_a, MK_b \in Z_r$ , and compute their respective public key,  $PK_a$  and  $PK_b$ , using the formula

$$PK_a = MK_a \cdot G \quad (2)$$

$$PK_b = MK_b \cdot G. \quad (3)$$

To encrypt a message  $M$ ,  $C_a$  randomly chooses an integer  $k \in Z_r$ .  $C_a$  computes the ciphertext with two parts, denoted as  $C_1$  and  $C_2$

$$C_1 = k \cdot G \quad (4)$$

$$C_2 = M + k \cdot PK_b. \quad (5)$$

Then  $C_a$  sends the ciphertext  $(C_1, C_2)$  to  $C_b$ . After receiving the ciphertext,  $C_b$  decrypts it using their private key

$$M = (C_2 + k \cdot PK_b) - C_1 \cdot MK_b. \quad (6)$$

Finally,  $C_b$  obtains the plaintext message  $M$  through the decryption process.

### B. Access Structure and Attribute

An access structure  $2^{\{F_1, F_2, \dots, F_n\}}$  comprises nonempty sets of attributes  $\{F_1, F_2, \dots, F_n\}$ . These sets within  $\Lambda$  denote authorized sets. The structure can be transformed into a Boolean formula and illustrated using an access tree. In this tree structure, attributes are represented by leaf nodes, whereas root or intermediate nodes symbolize logical operators (AND, OR). For instance, consider a scenario where access to decrypt a ciphertext is contingent upon possessing specific attributes. The access structure is expressed by the Boolean operation

$(A \wedge B) \vee (C \wedge D)$ . This expression implies that the entity must possess attributes  $A, B$  or  $C, D$  to decrypt the ciphertext successfully. Let  $S = \{A_1, A_2, \dots, A_n\}$  denote the attribute domain, where  $A_i$  represents the attribute set of client  $i$ . This set is denoted by an  $n$  bit string  $a_1, a_2, \dots, a_n$ , where  $a_i = 0$  if  $A_i \notin S$ , and  $a_i = 1$ , if  $A_i \in S$ . Similarly, an access policy  $\rho$  is defined based on the attributes within  $A$ , and can be represented by an  $n$ -bit string  $b_1, b_2, \dots, b_n$ , where  $b_i = 0$  if  $A_i \notin \rho$ , and  $b_i = 1$  if  $A_i \in \rho$ . For instance, consider a scenario, where  $S = \{A_1, A_2, A_4\}$  is the attribute set of the client and  $n = 5$ . In this case,  $\Lambda$  can be denoted as a 5-bit string 11010. Alternatively, for another example where  $n = 5$  and  $B = 00111$ , this indicates that possessing the attribute set  $A_3, A_4, A_5$  is essential for satisfying the access policy  $\rho$ .

### C. Linear Secret Sharing Scheme

In cryptography, a secret sharing scheme distributes a secret among a set of participants such that only authorized subsets can reconstruct it. The secret is divided into multiple shares, each of which is distributed to a different participant. Termed as an LSSS [32] over  $Z_r$ , this scheme conforms to some specific criteria.

- 1) All participating members  $F_i$  contribute secret shares forming a vector within  $Z_r$ .
- 2) Within this secret sharing scheme, a sharing matrix  $M$  of dimensions  $l \times n$  exists, where each row  $i \in \{1, 2, \dots, l\}$  corresponds to an element in the set  $F$ , retrievable through the function  $\rho(i)$ , mapping  $\{1, 2, \dots, l\}$  to  $F$ . Let  $\vec{v} = (s, v_2, v_3, \dots, v_n)$  denote a random vector, with  $s \in Z_r$  representing the shared secret message and  $v_2, v_3, \dots, v_n \in Z_r$  as arbitrary random numbers; then,  $(M \cdot \vec{v})_i \rightarrow \lambda_i$  partitions the secrets into  $l$  parts according to the LSSS.
- 3) For the specified LSSS to be valid, it must conform to the principle of linear reconstruction. Given an authorized set  $S$  in the access structure  $\Lambda$ , with  $I$  being a subset of  $\{1, 2, \dots, l\}$  and  $I$  defined as the set of indices  $i$  for which  $\rho(i)$  belongs to  $S$ , a set of constants  $c_i \in Z_r$  can be computed efficiently in polynomial time. The original secret  $s$  can be recovered by computing the linear combination  $\sum_{i \in I} c_i \lambda_i = s$ .

### D. Decisional Diffie-Hellman

The DDH problem on elliptic curves is formulated as follows: The challenger selects a cyclic group  $G_p$  of prime order  $r$ , where  $G$  represents a generator of the cyclic group,



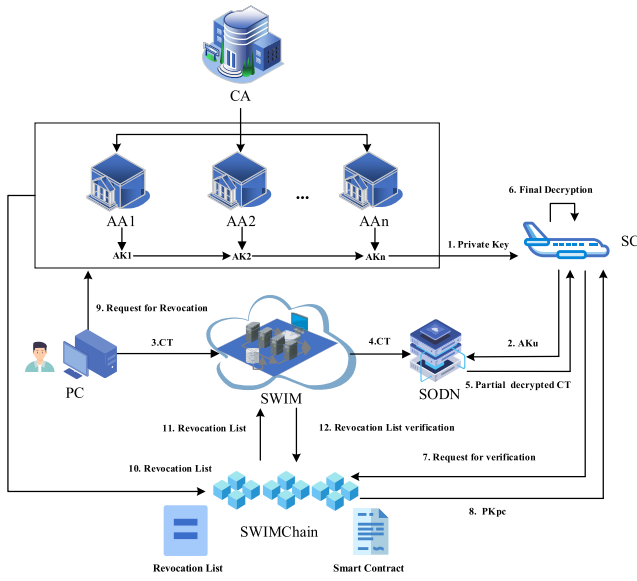


Fig. 2. Proposed scheme.

and selects two random exponents  $a$  and  $b$  from the set  $Z_r$ . The challenger then provides the adversary with a set of elements  $(G, aG, bG)$ . The task of the adversary is to distinguish between a legitimate element  $abG \in G_p$  and a random element  $R \in G_p$  given this tuple [16]. However, distinguishing between valid and random elements in polynomial time is difficult for an adversary.

There is a corresponding assumption for the above problem: It is challenging for distinguisher  $B$  to solve the DDH problem under the polynomial time limit. The advantage of  $B$  in solving the DDH problem on  $G_p$  is represented by the symbol  $\varepsilon$ . The absolute value of the difference between the probability of  $B$  correctly distinguishing  $abG$  and  $R$  does not exceed  $\varepsilon$

$$|Pr[B(G, aG, bG, Z = abG) = 0] - Pr[B(G, aG, bG, Z = R) = 0]| \leq \varepsilon. \quad (7)$$

The DDH assumption holds if  $B$ 's advantage in solving the problem in polynomial time is negligible.

#### IV. OVERVIEW OF THE PROPOSED SCHEME

In this section, we provide an overview of our scheme, including the system model, system definitions, and security model.

##### A. System Model

Fig. 2 illustrates our proposed CP-ABE scheme, which comprises the central authority (CA), SWIMChain, SWIM, the AA, the SWIM outsourcing decryption node (SODN), the SWIM data publish client (PC), and the SWIM data subscribe client (SC).

We present a detailed overview of each component as follows.

- 1) **CA:** The CA is a fully trusted entity that acts as the root of trust for the system. It initializes the system and generates the public parameters  $PP$  for the entire system.

- 2) **SWIMChain:** The SWIMChain stores the public key of the PC for trusted verification and maintains a revocation list to facilitate revocation.
- 3) **SWIM:** The SWIM is an information infrastructure consisting of servers, storage, routers, gateways, and other devices. It is the platform for data storage, publication, subscription, and sharing.
- 4) **AA:** The AA is a semitrusted entity responsible for attribute management. It operates independently to generate public-private key pairs for its respective attribute sets and distributes the corresponding attribute private key to the SC. Since each AA manages a different set of attributes and secret keys, the compromise of one AA does not affect the security of attributes managed by others.
- 5) **SODN:** The SODN has good storage and computing capabilities. It executes partial decryption operations and transmits the processed ciphertext to the SC for final decryption. The SODN is a semitrusted (honest-but-curious) node that only performs blind decryption and does not hold the private key. Availability attacks are outside the scope of this work.
- 6) **PC:** The PC generates and publishes data to the SWIM. These data include aeronautical, meteorological, and flight information collected from the aviation information management platform.
- 7) **SC:** The SC receives and uses the data published by the PC in the SWIM system. It can access the pre-decryption output from the SODN and execute the final decryption only if its attributes satisfy the access policy established by the PC.

##### B. System Definitions

This section explains the system variables and introduces the overall process. Nomenclature presents the symbols employed in this article.

The overall process of this system is introduced as follows.

- 1) **System Initialization** ( $q \rightarrow PP$ ): The CA selects a security parameter  $q$  and generates  $PP$  as its output.
- 2) **Key Generation** ( $UA_{i,GID}, MD_{i,GID}, GID, PP, t \rightarrow AK_{i,GID,t}$ ): The  $AA_i$  is responsible for generating and managing attribute keys for the attributes under its authority. The  $AA_i$  generates the epoch-bound temporary attribute key  $AK_{i,GID,t}$  by inputting the general attribute set  $UA_{i,GID}$ , the model attribute set  $MD_{i,GID}$ ,  $GID$  of the SC, the hourly epoch  $t$ , and  $PP$ .
- 3) **Data Encryption** ( $PP, PKA, M, (\Lambda, \rho), t, RL_{ver} \rightarrow CT$ ): The PC encrypts plaintext  $M$  under the access policy  $(\Lambda, \rho)$  using a KEM-DEM structure.
  - a) **KEM (CP-ABE Encryption):** The PC first runs CP-ABE to generate a ciphertext component  $(P, C_{1,x}, C_{2,x})$  under  $(\Lambda, \rho)$ , and derives a symmetric key  $K$ .
  - b) **DEM (Authenticated Encryption):** The PC employs public-key authenticated encryption with associated data (AEAD) as the DEM to encrypt  $M$  and achieve public-key verifiable integrity via the

publisher's signature, where  $AAD$  binds the hourly epoch  $t$  and the revocation list version number  $RL_{ver}$ .

- 4) *Predecryption*  $(PP, CT, AK_{i,GID,t}, (\Lambda, \rho)) \rightarrow CT'$ : The SODN receives the ciphertext and performs partial decryption of the ciphertext  $CT$ . It takes  $AK_{i,GID,t}$  and the ciphertext  $CT$  as inputs to output the predecrypted ciphertext  $CT'$ .
- 5) *Final Decryption*  $(CT') \rightarrow M$ : The SC decrypts the pre-decrypted ciphertext, inputs  $CT'$  and outputs the plaintext  $M$ .
- 6) *Attribute Revocation*  $GID \rightarrow (RL, RL_{ver})$ : The  $AA_i$  updates the SWIMChain revocation list  $RL$  to revoke the attributes by appending the SC's  $GID$  to  $RL$  and updates the corresponding  $RL_{ver}$ .
- 7) *Trusted Verification*  $(CT, PK_{PC}) \rightarrow \partial$ : The SC performs trusted verification using blockchain technology, including retrieving  $PK_{PC}$ . Finally, it calculates elliptic curve digital signature algorithm (ECDSA).  $Verify(PK_{PC}, (CT_K || nonce || AAD), CT_H)$  to generate  $\partial \in \{0, 1\}$  to confirm data integrity.

### C. Security Model

The security model is defined by a CCA game played between two roles, a challenger  $C$  and an adversary  $\mathcal{A}$  [33], [34], as formally described below.

*Initialization*:  $C$  generates  $PP$  of the system,  $MK$ , and  $PKA$ .  $C$  sends  $PKA$  to  $\mathcal{A}$ .

*Phase 1*:  $\mathcal{A}$  can make the following queries.

- 1) *Temporary Attribute Key Query*:  $\mathcal{A}$  adaptively requests temporary attribute key  $AK_{i,GID,t}$  for attribute sets of its choice.  $C$  generates and returns these temporary attribute keys. None of the queried attribute sets can satisfy the target access structure.
- 2) *Decryption Query*:  $\mathcal{A}$  submits an arbitrary ciphertext  $CT$  to the decryption oracle. The oracle first performs the trusted verification procedure. If verification fails, it returns  $\perp$ . Otherwise, it runs  $AEAD.Dec(K', nonce, CT_K)$ . If AEAD authentication fails, it returns  $\perp$ , or it returns the plaintext  $M$ .

*Challenge*:  $\mathcal{A}$  submits a challenge access structure  $(\Lambda, \rho)^*$  and selects two equal-length messages  $M_0$  and  $M_1$ .  $C$  randomly selects a bit  $\mu \in \{0, 1\}$  and runs the *Data Encryption*  $(PP, PKA, M, (\Lambda, \rho)^*, t^*, RL_{ver}^*)$  to encrypt  $M_\mu$  under the access structure  $(\Lambda, \rho)^*$  and the corresponding epoch  $t^*$  to generate ciphertext  $CT^*$ . Then  $C$  returns  $CT^*$  to  $\mathcal{A}$ .

*Phase 2*:  $\mathcal{A}$  continues to adaptively issue additional temporary attribute key queries and decryption queries, following the same constraints as in phase 1. In addition,  $\mathcal{A}$  cannot query the attribute set that satisfies the target access structure  $(\Lambda, \rho)^*$  during the key query phase,  $\mathcal{A}$  cannot query the challenge ciphertext  $CT^*$  during the decryption query phase.

*Guess*:  $\mathcal{A}$  outputs a guess  $\mu'$  for the bit  $\mu$ .

$\mathcal{A}$  wins the game if  $\mu' = \mu$ . The advantage of  $\mathcal{A}$  is defined as  $\text{Adv}_{\text{CPABE}, \mathcal{A}}^{\text{IND-CCA}}(1^\epsilon) = |\Pr[\mu' = \mu] - (1/2)|$  in this game.

This ensures that the ability of the adversary to distinguish between encryptions of  $M_0$  and  $M_1$  is rigorously quantified under IND-CCA with security parameter  $\xi$ .

## V. SCHEME CONSTRUCTION

In this section, we construct the proposed scheme, which comprises system initialization, key generation, data encryption, predecryption, final decryption, data verification, and attribute revocation.

### A. System Initialization

The CA selects a large prime number  $q$  to generate  $PP = \{GF(q), G, E, H, H_1\}$ , where  $GF(q)$  is a finite field,  $G$  is a generator,  $E$  is an elliptic curve over  $GF(q)$ , and  $H : \{0, 1\}^* \rightarrow Z_r^*$  is a hash function that maps the  $GID$  of the client to  $Z_r^*$ , and  $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^l$  is a cryptographic hash function that derives the symmetric key  $K$  from the concatenation of elliptic-curve elements and the access structure.

### B. Key Generation

The CA sends  $PP$  to the  $AA_i$  to generate  $PKA$  and  $MK$ .  $AA_i$  generates  $MK = \{y_i, k_i\}_{v_i}$  by randomly selecting integers  $y_i, k_i \in Z_r$ . It also generates a public key  $PKA = \{y_i G, k_i G\}_{v_i}$  for each attribute.

The attribute key  $AK'_{i,GID,t} = \{UK'_{i,GID,t}, MDK'_{i,GID,t}\}$  includes two parts: the universal attribute key  $UK'_{i,GID,t}$  and the model attribute key  $MDK'_{i,GID,t}$ , which is generated by  $AA_i$  and sent to the SC. The temporary attribute key  $AK_{i,GID,t} = \{UK_{i,GID,t}, MDK_{i,GID,t}\}$  includes two parts: the temporary universal attribute key  $UK_{i,GID,t}$  and the temporary model attribute key  $MDK_{i,GID,t}$ , which is generated by the SC and sent to the SODN.

The  $AA_i$  calculates the attribute key related to the  $GID$  by selecting  $m_i \in Z_r$  and the current hourly epoch  $t$  as follows:

$$UK'_{i,GID,t} = y_i + H(GID || t)k_i \quad (8)$$

$$MDK'_{i,GID,t} = y_i + H(GID || t)(k_i + m_i). \quad (9)$$

Then the SC chooses a random integer  $r \in Z_r$  and computes the temporary attribute key  $AK_{i,GID,t} = \{UK_{i,GID,t}, MDK_{i,GID,t}\}$  utilizing the formula as follows:

$$UK_{i,GID,t} = y_i + H(GID || t)k_i + r, \quad r \in Z_r \quad (10)$$

$$MDK_{i,GID,t} = y_i + H(GID || t)(k_i + m_i) + r, \quad r \in Z_r. \quad (11)$$

### C. Data Encryption

The PC executes the encryption procedure as outlined below.

First of all, the PC selects a random number  $s \in Z_r$  to calculate  $P$

$$P = s \cdot G. \quad (12)$$

Then the PC associates the ciphertext identifier  $ID_{md}$  with  $(\Lambda, \rho)$  and restricts  $ID_{md}$  by adding the valid model range  $MD = [MD_{AIXM}, MD_{FIXM}, MD_{IWXXM}]$  to the model attribute set  $MD_{i,GID}$ .

The PC chooses two random vectors  $\vec{v} = (s, v_1, \dots, v_m)$  and  $\vec{u} = (0, u_1, \dots, u_m)$ , computes  $\lambda_x = \Lambda_x \cdot \vec{v}$  and  $\omega_x = \Lambda_x \cdot \vec{u}$ , and calculates  $C_{1,x}$ , and  $C_{2,x}$

$$C_{1,x} = \lambda_x \cdot G + y_{\rho(x)} \cdot G \quad (13)$$

$$C_{2,x} = \begin{cases} \omega_x \cdot G + k_{\rho(x)} \cdot G, & \text{if } \rho(x) \in UA_{x,GID} \\ \omega_x \cdot G + (k_{\rho(x)} + m_{\rho(x)}) \cdot G, & \text{if } \rho(x) \in MD_{x,GID}. \end{cases} \quad (14)$$

After obtaining  $C_{1,x}$  and  $C_{2,x}$ , the PC concatenates  $P$  with  $C_{1,x}$ ,  $C_{2,x}$  and  $(\Lambda, \rho)$ , and inputs this concatenation into the hash function  $H_1$  to generate an independent encryption key  $K$

$$K = H_1(P \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)). \quad (15)$$

The PC derives the current hourly epoch  $t$  and a 96-bit *nonce*. It then constructs the associated data  $AAD$  and performs AEAD encryption on the plaintext data to obtain ciphertext  $CT_K$

$$nonce \leftarrow \{0, 1\}^{96} \quad (16)$$

$$AAD = (t \| RL_{ver} \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)) \quad (17)$$

$$CT_K = \text{AEAD.Enc}(K, nonce, M). \quad (18)$$

After that, the PC uses ECDSA to calculate  $CT_H$  for subsequent verification of data integrity and trustworthiness. It uses the private key  $SK_{PC}$  of PC to generate a cryptographic signature on  $(CT_K \| nonce \| AAD)$ , and verifies it using the corresponding public key  $PK_{PC}$

$$CT_H = \text{ECDSA.Sig}(SK_{PC}, (CT_K \| nonce \| AAD)). \quad (19)$$

Ultimately, the PC generates the ciphertext  $CT = \{(\Lambda, \rho), C_{1,x}, C_{2,x}, t, RL_{ver}, nonce, CT_K, CT_H\}$  and forwards it to the SWIM network.

#### D. Predecryption

The SODN performs the predecryption operation through the following steps to obtain the predecrypted ciphertext.

Based on the SC's attribute set  $A = \{A_1, A_2, \dots, A_n\}$ , the SODN generates a set  $X = \{x | \rho(x) \in A\}$ . When the SC's attribute satisfies the conditions of the access control policy  $(\Lambda, \rho)$ , a constant set  $\{c_x \in Z_r\}_{x \in X}$  can be identified within the polynomial duration, achieving  $\sum_{x \in X} c_x \Lambda_x = \varepsilon = (1, 0, \dots, 0)$ , indicating  $\sum_{x \in X} c_x \lambda_x = s$  and  $\sum_{x \in X} c_x \omega_x = 0$ .

The SODN computes the result  $D_x = C_{1,x} - AK_{\rho(x), GID, t} G + H(GID \| t) C_{2,x}$ . If  $\rho(x)$  is a normal attribute, then

$$\begin{aligned} D_x &= C_{1,x} - UK_{\rho(x), GID, t} G + H(GID \| t) C_{2,x} \\ &= \lambda_x G + y_{\rho(x)} G - (y_i + H(GID \| t) k_{\rho(x)} + r) G \\ &\quad + H(GID \| t) (\omega_x G + k_{\rho(x)} G) \\ &= \lambda_x G + H(GID \| t) \omega_x G - rG. \end{aligned} \quad (20)$$

If  $\rho(x)$  is a model attribute, then

$$\begin{aligned} D_x &= C_{1,x} - MDK_{\rho(x), GID, t} G + H(GID \| t) C_{2,x} \\ &= \lambda_x G + y_{\rho(x)} G \\ &\quad - (y_{\rho(x)} + H(GID \| t) (k_{\rho(x)} + m_{\rho(x)}) + r) G \end{aligned}$$

$$\begin{aligned} &+ H(GID \| t) (\omega_x G + (k_{\rho(x)} + m_{\rho(x)}) G) \\ &= \lambda_x G + H(GID \| t) \omega_x G - rG. \end{aligned} \quad (21)$$

After calculating  $D_x$ , the SODN calculates  $F_1$  and  $F_2$  as follows:

$$\begin{aligned} F_1 &= \sum_{x \in X} c_x D_x \\ &= \sum_{x \in X} c_x \lambda_x G + H(GID \| t) \sum_{x \in X} c_x \omega_x G \\ &\quad - r \sum_{x \in X} c_x G \\ &= sG - r \sum_{x \in X} c_x G \end{aligned} \quad (22)$$

$$F_2 = \sum_{x \in X} c_x G. \quad (23)$$

Then the SODN generates the predecrypted ciphertext  $CT' = \{(\Lambda, \rho), C_{1,x}, C_{2,x}, t, RL_{ver}, nonce, CT_K, CT_H, F_1, F_2\}$  and transmits it to the SC for final decryption.

#### E. Final Decryption

After the predecryption process, the SC performs the final decryption to obtain the plaintext.

After the SC obtains the predecrypted ciphertext  $CT' = \{(\Lambda, \rho), C_{1,x}, C_{2,x}, t, RL_{ver}, nonce, CT_K, CT_H, F_1, F_2\}$  from the SODN, it performs a trusted verification on the ciphertext, as shown in Section V-F. After ensuring the ciphertext source is trustworthy and its integrity is not compromised, the SC calculates  $P'$

$$\begin{aligned} P' &= F_1 + r \cdot F_2 \\ &= sG + r \sum_{x \in X} c_x G - r \sum_{x \in X} c_x G \\ &= sG. \end{aligned} \quad (24)$$

After generating  $P'$ , the SC concatenated  $P'$  with  $C_{1,x}$ ,  $C_{2,x}$ , and  $(\Lambda, \rho)$ , and this concatenation result is input into the hash function  $H_1$  to generate the encryption key  $K'$

$$K' = H_1(P' \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)). \quad (25)$$

Finally, if  $K' = K$ , the SC reconstructs the associated data  $AAD$  and decrypts the ciphertext using AEAD as follows:

$$AAD = (t \| RL_{ver} \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)) \quad (26)$$

$$M = \text{AEAD.Dec}(K', nonce, CT_K). \quad (27)$$

#### F. Data Verification

Our scheme uses the immutable properties of blockchain to achieve trusted verification of decrypted data. The role of blockchain in verification is to ensure the authenticity of public keys. Signatures are generated using ECDSA, and the verification process consists of three steps: retrieving the public key from the blockchain, performing the verification calculation, and determining legitimacy. First, after obtaining the ciphertext  $CT_K$  and the ECDSA signature  $CT_H$ , the SC requests the public key  $PK_{PC}$  from the SWIMChain.

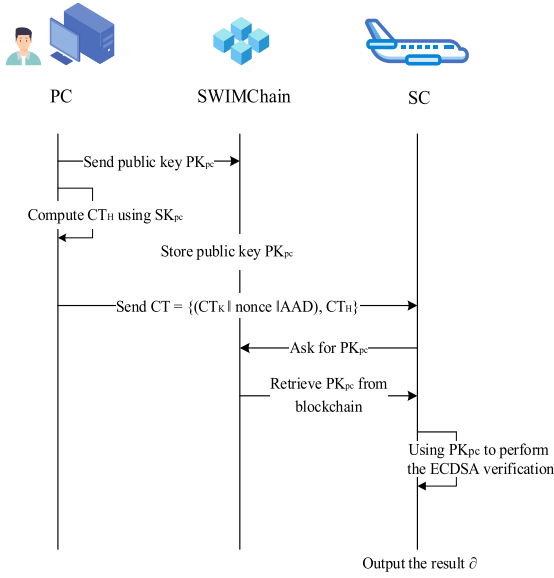


Fig. 3. Data verification process.

Then the SC uses  $PK_{PC}$ ,  $CT_K$ ,  $nonce$ ,  $AAD$ , and  $CT_H$  to calculate ECDSA.  $\text{Verify}(PK_{PC}, (CT_K || nonce || AAD), CT_H)$ .

After that, the SC performs a validity check and outputs  $\delta$ . If  $\delta = 1$ , it proves the trustworthiness and integrity of the ciphertext, and the SC starts to decrypt the ciphertext  $CT_K$ . The SWIMChain-assisted verification effectively guarantees the integrity and trustworthiness of the data. The data verification process is shown in Fig. 3.

### G. Attribute Revocation

Upon receiving a revocation request from the PC,  $AA_i$  revokes the SC by inserting its  $GID_{SC}$  into the revocation list  $RL$ . Then,  $AA_i$  updates  $RL$  on the SWIMChain, where the updated list is immutably recorded to prevent tampering. For each update, the SWIMChain also outputs a revocation-list version number  $RL_{ver}$ , which uniquely identifies the latest revocation state. After SWIM obtains the updated  $(RL, RL_{ver})$ , it checks whether the SC's  $GID_{SC}$  appears in  $RL$ . If  $GID_{SC} \in RL$ , SWIM refuses to distribute subsequent ciphertexts to the corresponding SODN.

In addition, to support time-bound access with hourly epochs, once a SC is revoked at an epoch  $t_{rev}$ ,  $AA_i$  stops issuing the epoch-bound key components for that  $GID_{SC}$  for all future epochs  $t' \geq t_{rev}$ . As a result, even if a revoked SC has cached historical ciphertext, it cannot derive the required epoch key updates for later epochs and thus cannot decrypt ciphertext generated after revocation. The attribute revocation process is shown in Algorithm 1.

## VI. SECURITY ANALYSIS

In this section, we analyze the scheme's security, focusing on CCA security, anticollusion attack, and forward security.

### A. CCA Security

Our scheme rests on the assumption of DDH for its security. In this section, we prove that our scheme achieves IND-CCA security.

### Algorithm 1 Attribute Revocation

**Input:**  $GID_{SC}, RL, t', t_{rev}, RL_{ver}$

**Output:** Revocation status in SWIM

```

1  The PC sends a request to the  $AA_i$  to revoke SC
2   $t_{rev} \leftarrow \lfloor \frac{UTC_{time}}{3600} \rfloor$  // Record the revocation epoch
3   $RL \leftarrow RL \cup (GID_{SC}, t_{rev})$ 
4  The  $AA_i$  updates the SWIMChain revocation list  $RL$  and obtains the latest  $RL_{ver}$ 
5  The SWIMChain sends the updated  $(RL, RL_{ver})$  to the SWIM
6  If  $GID_{SC}$  is in  $RL$  then
7  // The SC corresponding to this  $GID_{SC}$  is revoked
8  For each key-update request  $(GID_{SC}, t')$ 
9  if  $t' \geq t_{rev}$  then
10     return true
11  else
12     return false
13  end if
14 else //  $GID_{SC}$  is not in the revocation list
15     return false
16 end if

```

**Theorem 1:** Assuming the DDH assumption holds, then the scheme proposed in this article is secure under the chosen ciphertext attack.

**Proof:** We conduct the security proof in the random-oracle model, where the hash functions  $H$  and  $H_1$  are modeled as random oracles. Let  $G_1$  be a cyclic group of prime order  $p$ , with generator  $G$ . The challenger  $C$  randomly selects  $a, b \in \mathbb{Z}_r$  to compute  $abG$ , and then randomly selects  $\mu \in \{0, 1\}$ , if  $\mu = 0$ , let  $Z = abG$ ; if  $\mu = 1$ , randomly selects  $R \in G$  and lets  $Z = R$ .

Then  $C$  sends the DDH instance  $(G, aG, bG, Z)$  to distinguisher  $B$ .  $B$ 's goal is to determine whether  $R$  is  $abG$  or a random element in  $G_1$ .  $B$  outputs a bit  $e' \in \{0, 1\}$  as a judgment on the original challenge bit  $e$ .

**Initialization:** During the initialization phase of the game,  $B$  generates  $PP$ , then randomly selects two numbers  $y_i, k_i \in \mathbb{Z}_r$  to construct  $MK = \{y_i, k_i\}$ , generates  $PKA = \{y_i G, k_i G\}$  for each attribute  $i$  according to  $MK$ , and finally,  $B$  sends  $PKA$  to  $\mathcal{A}$ .

**Phase 1:**  $\mathcal{A}$  can make the following queries.

- 1)  $H$  Query: For the  $i$ th query to  $H$ ,  $B$  uniformly selects a random value  $q_i$  from  $\mathbb{Z}_r$  and sends it to  $\mathcal{A}$ .
- 2)  $H_1$  Query: For the  $i$ th query to  $H_1$ ,  $B$  uniformly selects a random value  $\vartheta_i$  from  $\{0, 1\}^l$  and sends it to  $\mathcal{A}$ .
- 3) **Temporary Attribute Key Query:** Assume that  $\mathcal{A}$  can query any attribute key. However, there is a restriction that the attribute key obtained by  $\mathcal{A}$  cannot meet the conditions set in the access structure  $(\Lambda, \rho)^*$ .

If the set of attributes of  $\mathcal{A}$  does not satisfy  $(\Lambda, \rho)^*$ ,  $\mathcal{A}$  requests to generate a temporary attribute key  $AK_{i, GID, t}$  using a certain attribute set, and the requested attribute set cannot meet the target access structure.  $B$  searches  $q_i$  from the hash list of  $H$  and randomly selects  $r \in \mathbb{Z}_r$  to generate and return the temporary attribute key  $AK_{i, GID, t} = \{UK_{i, GID, t}, MDK_{i, GID, t}\}$  of these attribute sets.



$B$  calculates  $UK_{i,GID,t}$  and  $MDK_{i,GID,t}$  as follows:

$$UK_{i,GID,t} = y_i + q_i k_i + r \quad (28)$$

$$MDK_{i,GID,t} = y_i + q_i (k_i + m_i) + r. \quad (29)$$

$E_1$ : If  $\mathcal{A}$  does not make a hash query to  $H$  on any input that satisfies the given formula.

When event  $E_1$  occurs,  $B$  aborts and returns  $\perp$  to  $\mathcal{A}$ . If none of the abort conditions are met,  $B$  returns the results  $AK_{i,GID,t}$  to  $\mathcal{A}$ .

4) *Decryption Query*:  $\mathcal{A}$  sends the ciphertext  $CT = \{(\Lambda, \rho), C_{1,x}, C_{2,x}, t, RL_{ver}, nonce, CT_K, CT_H\}$  to  $B$  for decryption queries.

It first performs trusted verification. If verification fails, it outputs  $\perp$ . Otherwise, it proceeds to reconstruct the encryption key.

The SODN computes the predecryption result according to (20)–(23).

The SC concatenates  $P$  with  $C_{1,x}$ ,  $C_{2,x}$ , and  $(\Lambda, \rho)$  to generate the encryption key  $K$  and decrypt  $CT_K$  to obtain  $M$

$$K = \vartheta_i = H_1(P \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)) \quad (30)$$

$$AAD = (t \| RL_{ver} \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)) \quad (31)$$

$$M = \text{AEAD.Dec}(K, nonce, CT_K) \quad (32)$$

where  $B$  checks if the above formula is satisfied for  $\vartheta_i$  and performs the AEAD decryption in (32) to return  $M$  or  $\perp$ . If a hash query satisfies the above formula,  $B$  returns the result to  $\mathcal{A}$  as the decryption query for the ciphertext  $CT$ .

$E_2$ : If  $\mathcal{A}$  does not make a hash query to  $H_1$  on any input that satisfies the given formula.

When event  $E_2$  occurs,  $B$  aborts and returns  $\perp$  to  $\mathcal{A}$ .

*Challenge*:  $\mathcal{A}$  selects two equidistant-length messages  $M_0$  and  $M_1$  for  $B$ . After that,  $B$  selects a random bit  $\mu \in \{0, 1\}$ , and encrypts the corresponding message  $M_\mu$  according to the access structure  $(\Lambda, \rho)^*$ . At the same time,  $B$  arbitrarily selects two vectors  $\vec{v} = (s, v_1, \dots, v_m)$  and  $\vec{u} = (0, u_1, \dots, u_m)$ , and calculates  $\lambda_x = \Lambda_x \cdot \vec{v}$  and  $\omega_x = \Lambda_x \cdot \vec{u}$ , and uses the  $Z$  received from  $C$ . Then,  $B$  calculates

$$P = s \cdot G \quad (33)$$

$$C_{1,x} = \lambda_x \cdot aG + y_{\rho(x)} Z \quad (34)$$

$$C_{2,x} = \begin{cases} \omega_x \cdot G + k_{\rho(x)} \cdot bG, & \text{if } \rho(x) \in UA_{x,GID} \\ \omega_x \cdot G + (k_{\rho(x)} + m_{\rho(x)}) \cdot bG, & \text{if } \rho(x) \in MD_{x,GID} \end{cases} \quad (35)$$

$$K^* = H_1(P \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)) \quad (36)$$

$$AAD^* = (t^* \| RL_{ver}^* \| C_{1,x} \| C_{2,x} \| (\Lambda, \rho)^*) \quad (37)$$

$$CT_K^* = \text{AEAD.Enc}(K^*, nonce^*, M_\mu). \quad (38)$$

Then  $B$  calculates the challenge ciphertext  $CT^* = \{(\Lambda, \rho)^*, C_{1,x}, C_{2,x}, t^*, RL_{ver}^*, nonce^*, CT_K^*, CT_H^*\}$  and sends it to  $\mathcal{A}$ .

*Phase 2*:  $\mathcal{A}$  continues to make queries with the same constraints as in phase 1, except in the following situations.

1)  $\mathcal{A}$  still cannot use the attribute set that satisfies the target access structure  $(\Lambda, \rho)^*$ .

2)  $\mathcal{A}$  cannot query the challenge ciphertext  $CT^*$  during the decryption query phase.

*Guess*:  $\mathcal{A}$  submits the guessed bit  $\mu' \in \{0, 1\}$ . If  $\mu' = \mu$ , then  $B$  outputs  $e' = 0$ ; otherwise,  $B$  outputs  $e' = 1$ . Whether  $B$  can successfully output  $e = e'$  is the key to solving the DDH problem.

Next, we analyze the probability of the abort event. For  $E_1$ , suppose that  $\mathcal{A}$  performs  $q_H$  queries of size  $H$ . The output space of  $H$  is  $Z_r$ , with a size of  $r$ . Therefore, the probability of  $E_1$  occurring is defined as follows:

$$\Pr[E_1] \leq \frac{q_H}{p}. \quad (39)$$

For  $E_2$ , suppose that  $\mathcal{A}$  performs  $q_{H1}$  queries of size  $H_1$ . The output space of  $H_1$  is  $\{0, 1\}_l$ , with a size of  $l$ . Therefore, the probability of  $E_2$  occurring is defined as follows:

$$\Pr[E_2] \leq \frac{q_{H1}}{2^l}. \quad (40)$$

Therefore, the probability of the abort event is

$$\begin{aligned} \Pr[E] &= \Pr[E_1 \vee E_2] \\ &\leq \Pr[E_1] + \Pr[E_2] \\ &\leq \frac{q_H}{p} + \frac{q_{H1}}{2^l} \\ &= \text{negl}(\xi). \end{aligned} \quad (41)$$

If none of the abort events  $E_1$  and  $E_2$  occur, and the temporary attribute key queries and decryption queries are valid, then  $\mathcal{A}$  can distinguish between the real and the simulated experiment with nonnegligible advantage.

Then, we analyze the advantage of  $B$  in solving the DDH problem. Under the DDH assumption, the distributions of  $abG$  and a random group element  $R$  are computationally indistinguishable. Consider the two possible cases for the bit  $e$ .

- 1) *Case 1* ( $e=0$ ): In this case,  $Z = abG$ , and the challenge ciphertext is generated exactly according to the real encryption algorithm.  $C_{1,x}$  embedded in the challenge ciphertext is correctly formed,  $\mathcal{A}$ 's advantage in attacking is  $\varepsilon_a = \Pr[\mu' = \mu] - (1/2)$ , then, the probability of  $B$  winning is  $\Pr[e' = e | e = 0 \wedge \bar{E}] = \Pr[\mu' = \mu | e = 0 \wedge \bar{E}] = \varepsilon_a + (1/2)$ .
- 2) *Case 2* ( $e=1$ ): In this case,  $Z = R$ , which is uniformly random in  $G_1$ . All components of the challenge ciphertext that are derived from  $Z$  are distributed independently of the challenge bit  $\mu$ , so the challenge ciphertext reveals no information about  $\mu$ , and  $\mathcal{A}$  cannot do better than random guessing. Then, the probability of  $B$  winning is  $\Pr[e' = e | e = 1 \wedge \bar{E}] = \Pr[\mu' = \mu | e = 1 \wedge \bar{E}] = (1/2)$ .

$B$  cannot output the correct result when meeting an abort situation. In this case,  $B$  does not obtain any useful information from  $\mathcal{A}$ 's output and therefore outputs a uniformly random guess for  $e$ , and it can be considered that  $\Pr[e' = e | E] = (1/2)$ , so according to the law of total probability, we can calculate that

$$\Pr[e' = e] = \Pr[e' = e | \bar{E}] \Pr[\bar{E}] + \frac{1}{2} \Pr[E] \quad (42)$$

where  $e$  is a randomly selected bit, so  $\Pr[e = 0] = \Pr[e = 1] = (1/2)$ . The advantage of  $B$  to solve the DDH problem is defined as follows:

$$\begin{aligned}
\text{Adv}_B &= \left| \Pr[e' = e] - \frac{1}{2} \right| \\
&= \Pr[e' = e | \bar{E}] \Pr[\bar{E}] + \frac{1}{2} \Pr[E] - \frac{1}{2} \\
&= \Pr[e' = e | (e = 0) \wedge \bar{E}] \Pr[e = 0 | \bar{E}] \Pr[\bar{E}] \\
&\quad + \Pr[e' = e | (e = 1) \wedge \bar{E}] \Pr[e = 1 | \bar{E}] \Pr[\bar{E}] \\
&\quad + \frac{1}{2} \Pr[E] - \frac{1}{2} \\
&= \Pr[\mu' = \mu | (e = 0) \wedge \bar{E}] \Pr[e = 0] \\
&\quad + \Pr[\mu' = \mu | (e = 1) \wedge \bar{E}] \Pr[e = 1] + \frac{1}{2} \Pr[E] - \frac{1}{2} \\
&= \left( \frac{\varepsilon_a}{2} + \frac{1}{2} \right) (1 - \Pr[E]) + \frac{1}{2} \Pr[E] - \frac{1}{2} \\
&= \frac{\varepsilon_a}{2} (1 - \text{negl}(\xi)). \tag{43}
\end{aligned}$$

Utilizing the additivity of probability, we partition  $\Pr[(e' = e) \wedge \bar{E}]$  based on the value of the challenge bit  $e \in \{0, 1\}$ , obtaining

$$\begin{aligned}
\Pr[(e' = e) \wedge \bar{E}] &= \Pr[(e' = e) \wedge \bar{E} \wedge (e = 0)] \\
&\quad + \Pr[(e' = e) \wedge \bar{E} \wedge (e = 1)]. \tag{44}
\end{aligned}$$

Subsequently, by dividing both sides of the equation by the term  $\Pr[\bar{E}]$ , we can obtain the following formula:

$$\begin{aligned}
&\frac{\Pr[(e' = e) \wedge \bar{E}]}{\Pr[\bar{E}]} \\
&= \frac{\Pr[e' = e | (e = 0) \wedge \bar{E}] \Pr[(e = 0) \wedge \bar{E}]}{\Pr[\bar{E}]} \\
&\quad + \frac{\Pr[e' = e | (e = 1) \wedge \bar{E}] \Pr[(e = 1) \wedge \bar{E}]}{\Pr[\bar{E}]} \tag{45}
\end{aligned}$$

$$\begin{aligned}
&\Pr[(e' = e) | \bar{E}] \\
&= (\Pr[e' = e | (e = 0) \wedge \bar{E}] \Pr[(e = 0) | \bar{E}]) \\
&\quad + (\Pr[e' = e | (e = 1) \wedge \bar{E}] \Pr[(e = 1) | \bar{E}]). \tag{46}
\end{aligned}$$

If the advantage  $\varepsilon_a$  of the adversary  $\mathcal{A}$  in the IND-CCA experiment is nonnegligible, so would be the advantage of  $B$  in solving the DDH problem, which contradicts the DDH assumption. Therefore, our scheme achieves IND-CCA security.

In addition to the formal IND-CCA reduction proof, we further clarify why chosen ciphertext attacks are infeasible in practical deployments. In our proposed scheme, while an attacker may seemingly initiate decryption queries, this path is effectively blocked at the protocol layer by the following mechanisms.

Specifically, before executing decryption, the SC performs trusted verification using the PC's signature and the public key obtained from SWIMChain. Thus, any tampering with the ciphertext components will be rejected, resulting in an output of  $\perp$ . Since the derived key  $K$  and AAD are tightly

bound to the ciphertext and the access structure, any alteration modifies the derived key, resulting in  $\perp$  instead of a meaningful oracle response. Moreover, outsourced decryption only executes predecryption computation at SODN, while the final AEAD decryption is completed at the SC after verification, which prevents outsourcing from introducing an additional decryption oracle.

By these mechanisms, the system prevents adversaries from obtaining a useful adaptive decryption oracle; therefore, our scheme achieves IND-CCA security.

### B. Anticollusion Attack

To achieve collusion resistance, we tightly bind user attributes to their respective  $GID$  during key generation. With this design, even if different clients try to decrypt by collaboratively sharing their own attribute keys, their keys cannot be effective in combination because each key is deeply associated with the  $GID$ . The ciphertext can be decrypted only if at least one user independently satisfies the established access policy, under which the key unbinding conditions are fulfilled.

For instance, client  $P$  aims to collaborate with client  $Q$  to decrypt the  $CT$ . According to the access policy  $(A \wedge B) \vee (C \wedge D)$ , where  $P$  only possesses attributes  $A$  and  $C$ , and  $Q$  only has attributes  $B$  and  $D$ , neither of them can decrypt the ciphertext individually. Upon collusion,  $P$  may only obtain a portion of the predecryption ciphertext  $D_P$  from

$$D_P = \lambda_x G + H(GID_P || t) \omega_x G - rG. \tag{47}$$

Similarly,  $Q$  can only access part of the ciphertext  $D_Q$  from

$$D_Q = \lambda_x G + H(GID_Q || t) \omega_x G - rG. \tag{48}$$

As each client has a unique  $GID$  in the system, denoted as  $GID_P \neq GID_Q$ ,  $P$  and  $Q$  cannot collaborate to determine  $\{c_x \in \mathbb{Z}_r\}_{x \in X}$  and satisfy  $\sum_{x \in X} c_x \Lambda_x = (1, 0, \dots, 0)$ . Therefore, decryption cannot be performed normally.

Regarding the possibility of collusion between the SODN and clients, the SODN only holds publicly accessible system parameters and does not hold any client's private information. Even if the SODN colludes with client  $P$ , it cannot disrupt the decryption process bound to  $GID$  because there exists a term  $H(GID_P || t)$  that acts as a client-specific blinding factor, which the SODN cannot infer. Furthermore, the primary role of SODN is to perform the predecryption process, but it does not participate in the final decryption step. Final decryption requires the client's private key and cannot be performed without it. Therefore, the above collusion attack methods are ineffective, and our solution achieves anticollusion capability.

Beyond user collusion, we further analyze the security under authority compromise.

If a single  $AA_i$  is compromised, the adversary learns the secret key  $\{y_i, k_i\}$  held by that authority. As a result, any access policy whose satisfaction relies on attributes issued by  $AA_i$  becomes insecure. However, compromising  $AA_i$  does not affect attributes managed by other honest authorities  $AA_j (j \neq i)$ . When a ciphertext is encrypted under an access policy that spans multiple authorities, a compromised  $AA_i$  cannot produce valid key components corresponding to  $AA_j$ . Therefore, the

TABLE II  
TIME COST OF MAIN OPERATIONS

| Operation                              | Time Cost (ms) |
|----------------------------------------|----------------|
| Elliptic curve addition in $G_1$       | 0.00128        |
| Elliptic curve multiplication in $G_1$ | 0.0670         |
| Hashing                                | 0.00283        |

ciphertext remains confidential as long as the access structure requires at least one attribute issued by an uncompromised authority.

If multiple attribute authorities collude, they may share their attribute secret keys. They can decrypt a ciphertext only when the union of their issued attributes satisfies the ciphertext's access policy. They cannot forge keys for noncolluding authorities, since each authority independently generates and retains its own secret key  $\{y_i, k_i\}$ .

### C. Forward Security

Let  $t$  denote the hourly epoch and suppose a client with identity  $GID$  is revoked at an epoch  $t_{rev}$ . In our construction, the epoch-bound attribute key components are derived using  $H(GID||t)$ , and  $AA_i$  stops issuing the updated key material for all epochs  $t \geq t_{rev}$  once revocation occurs. Considering any ciphertext generated at epoch  $t \geq t_{rev}$ , decryption requires recovering  $K$  from  $CT$  using the valid epoch-bound keys for the same  $t$ . Since the revoked client cannot obtain the necessary key updates after  $t_{rev}$ , it cannot derive the correct  $K$ , and thus cannot successfully decrypt the  $CT_K$ . Therefore, a revoked client cannot decrypt ciphertexts created after the revocation time. This implies that the scheme provides forward security with respect to revocation.

## VII. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our scheme and compare it with existing schemes, including PFCPABE [16], EECPABE [19], LACMMAA [21], and EIMACPABE [22], in terms of security features, communication overhead, and computational overhead. In addition, we conduct experimental simulations to verify the effectiveness of our scheme.

### A. Experimental Environment

We conduct the simulation experiment on a Dell Precision 7540 workstation featuring an Intel<sup>1</sup> Core<sup>2</sup> i7-9850H processor (2.60 GHz) and 32 GB RAM, with Ubuntu 22.04 LTS as the operating system, running in a virtual machine. Our scheme is implemented in Python, and we employ Charm-Crypto (version 0.50) to implement elliptic curve operations. For the symmetric encryption part, we adopted the AEAD scheme ChaCha20-Poly1305, using the cryptography library in Python to implement. In the experiment, we select NIST P-256 (secp256r1) as the curve. This curve is defined over a 256-bit prime field and has an elliptic curve group of order

<sup>1</sup>Registered trademark.

<sup>2</sup>Trademarked.

256 bits, corresponding to approximately 128 security level. Table II shows the time cost of the main operations.

Our scheme performs the attribute revocation and data verification process on the blockchain. We experimentally evaluate the average latency of the revocation and data verification-related smart contracts under different transactions per second (TPS) settings with a total of 1000 submitted transactions. Table III shows the test environment parameters.

### B. Security Comparison

Table IV summarizes the security features of our scheme and other schemes. The PFCPABE, EECPABE, LACMMAA, and EIMACPABE schemes use an ECC-based pairing-free method and a more expressive and fine-grained LSSS to build access policies. In terms of architecture, the PFCPABE scheme uses a single AA to generate keys, which introduces the risk of key escrow and single-point failure. In contrast, the EECPABE scheme, LACMMAA scheme, EIMACPABE scheme, and our LMDA scheme all use a multiauthority architecture, which effectively mitigates the key-escrow problem and is better suited to the requirements of SWIM systems. In terms of functionality, existing schemes exhibit notable limitations. PFCPABE does not support dynamic attributes or outsourced decryption; LACMMAA lacks support for dynamic attributes and attribute revocation; EIMACPABE does not support dynamic attributes, outsourced decryption, or attribute revocation. In contrast, our scheme supports all of these security features. With respect to verifiability, most schemes do not support blockchain or outsourced verification. Our scheme uniquely integrates blockchain technology, ensuring outsourced verification of computation results. Meanwhile, our scheme achieves CCA security, whereas the other schemes provide only chosen plaintext attack (CPA) security.

### C. Communication Overhead

Table V presents a comparison of communication overhead between our scheme and existing schemes. To ensure uniformity in the comparison, we assume that all schemes operate at the same security level and use the basic operation based on the 256-bit elliptic curve as the comparison unit, represented by the symbol " $S$ ." We represent the size of a point on this curve as " $2S$ ," establishing it as the unified benchmark for calculating communication overhead. We denote the total number of attributes as " $A$ ," and let " $L$ " represents the number of leaf nodes in the access structure, which is equivalent to the row count in the corresponding access matrix. The number of the receiver's attributes is defined as " $R$ ," indicating the minimum attribute set required to satisfy the access structure.

Compared with the LACMMAA scheme, our scheme incurs a slightly higher communication overhead in the public key part and ciphertext part. In the public key part, the overhead results from the introduction of dynamic attributes, which enable more fine-grained access control in SWIM. In the ciphertext part, the overhead mainly results from incorporating an AEAD layer, which requires transmitting a nonce and an authentication tag, as well as carrying the associated data to enable authenticated decryption and time-bound access control

TABLE III  
PARAMETERS OF SMART CONTRACT PERFORMANCE TESTING ENVIRONMENT

| Tool                | Environment        | Version  | Parameters                    |
|---------------------|--------------------|----------|-------------------------------|
| Blockchain          | Fabric             | 2.2.4    | 2 peers, the batch time is 2s |
| Smart Contract      | Go                 | 1.17.13  |                               |
| Node                | Docker             | 20.10.21 |                               |
| Consensus Algorithm | Raft               |          |                               |
| Test Tool           | Caliper-benchmarks | 0.4.2    | 2 workers                     |

TABLE IV  
SECURITY FEATURES COMPARISON

| Scheme    | MA | DA | DO | VO | BC | RV | CCA |
|-----------|----|----|----|----|----|----|-----|
| PFCPABE   | ×  | ×  | ×  | ×  | ×  | √  | ×   |
| EECPABE   | √  | √  | √  | ×  | ×  | √  | ×   |
| LACMMAA   | √  | ×  | √  | ×  | ×  | ×  | ×   |
| EIMACPABE | √  | ×  | ×  | ×  | √  | ×  | ×   |
| ours      | √  | √  | √  | √  | √  | √  | √   |

Note: MA: Multiauthority, DA: Dynamic Attributes, DO: Decryption Outsourcing, VO: Verification Outsourcing, BC: Blockchain, RV: Revocation, CCA: IND-CCA Security.

TABLE V  
COMMUNICATION OVERHEAD COMPARISON

| Scheme    | Public Key  | Private Key | Ciphertext  |
|-----------|-------------|-------------|-------------|
| PFCPABE   | $(2A + 2)S$ | $(R)S$      | $(6L)S$     |
| EECPABE   | $(2A + 2)S$ | $(R)S$      | $(3L + 1)S$ |
| LACMMAA   | $(2A + 1)S$ | $(R)S$      | $(2L + 1)S$ |
| EIMACPABE | $(2A + 2)S$ | $(R)S$      | $(6L)S$     |
| ours      | $(2A + 2)S$ | $(R)S$      | $(2L + 2)S$ |

in SWIM. Therefore, the additional cost is acceptable. In contrast, the private key size of the proposed scheme remains unchanged, and its ciphertext communication overhead is better than that of other comparison schemes that exclude the LACMMAA scheme.

#### D. Computational Overhead

Table VI presents a comparison of computational overhead between our scheme and other schemes. The setup, encryption, predecryption, and decryption stages primarily account for the computational overhead involving a series of hash and scalar multiplication operations. For ease of comparison, we select scalar multiplication operations as the unit of measurement for computational overhead and assign them the symbol “G.”

During the setup phase, the computational overhead of the PFCPABE scheme is slightly lower than that of our scheme. This is because our scheme requires the configuration of multiple attribute authorities during initialization, whereas PFCPABE only needs to set up a single authority, resulting in lower initialization costs. Compared with the LACMMAA scheme, our scheme introduces dynamic attributes as model attributes in the encryption process, resulting in additional scalar multiplication operations. Nevertheless, this extra overhead is acceptable, as it helps achieve more fine-grained access

control within the SWIM system. In addition, the EECPABE, LACMMAA scheme, and our scheme all employ outsourced decryption, and the SC only needs to perform one scalar multiplication to complete the decryption, thereby significantly reducing the final decryption overhead.

Fig. 4 presents a comparison of key generation time between our scheme and the other schemes under different numbers of attributes. Since all schemes are CP-ABE-based and do not require combining access policies with keys, the computational overhead differences among these schemes in the key generation process are insignificant.

Fig. 5 presents a comparison of encryption time between our scheme and the other schemes under different numbers of attributes. Compared with the PFCPABE and EECPABE schemes, our scheme achieves a significantly lower encryption time. Since the EECPABE scheme involves more scalar multiplication operations and considers time and location attributes during the encryption process, it results in greater overhead than the other three schemes. Moreover, our scheme introduces the SWIM model domain as a dynamic attribute within the access structure. Although this results in a slightly higher encryption time than that of the LACMMAA scheme, the additional overhead is justified, as it enables more fine-grained access control.

Fig. 6 presents a comparison of decryption time between our scheme and other schemes under different attribute numbers. Compared with the PFCPABE scheme, which performs the entire decryption operation locally, the other schemes leverage outsourced decryption to reduce the decryption burden on the user side. In these schemes, most of the complex operations are completed by the outsourcing decryption node. Meanwhile, the SC can finalize the decryption with only a single scalar multiplication, thereby significantly reducing the decryption overhead.

Fig. 7 presents a comparison of total execution time overhead between our scheme and other schemes under different



TABLE VI  
COMPUTATIONAL OVERHEAD COMPARISON

| Scheme    | Setup       | Encryption  | Predecryption | Decryption |
|-----------|-------------|-------------|---------------|------------|
| PFCPABE   | $(A + 1)G$  | $(4L + 1)G$ | -             | $(R + 1)G$ |
| EECPABE   | -           | $(4L + 1)G$ | $(R + 1)G$    | $(1)G$     |
| LACMMAA   | $(2A + 1)G$ | $(3L + 1)G$ | $(R)G$        | $(1)G$     |
| EIMACPABE | $(2A + 1)G$ | $(4L + 1)G$ | -             | $(R + 1)G$ |
| ours      | $(2A + 1)G$ | $(4L + 1)G$ | $(R)G$        | $(1)G$     |

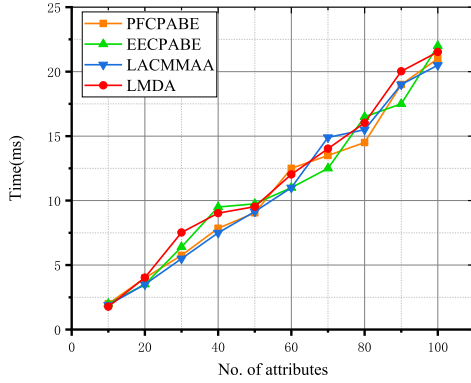


Fig. 4. Comparison of key generation time overhead.

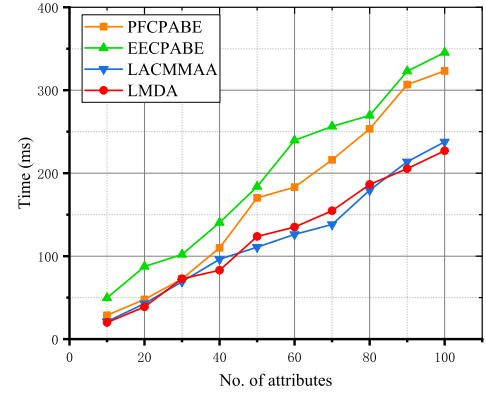


Fig. 7. Comparison of total execution time overhead.

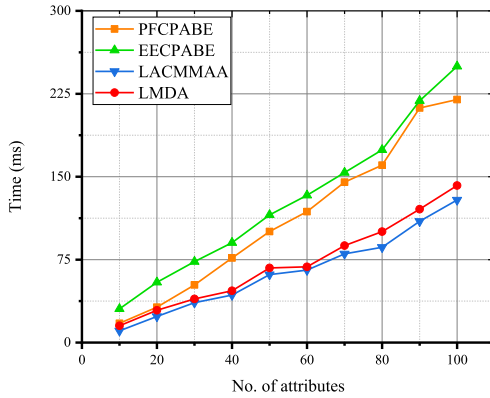


Fig. 5. Comparison of encryption time overhead.

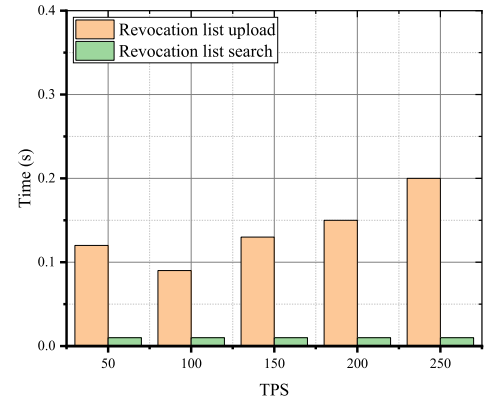


Fig. 8. Average latency of attribute revocation.

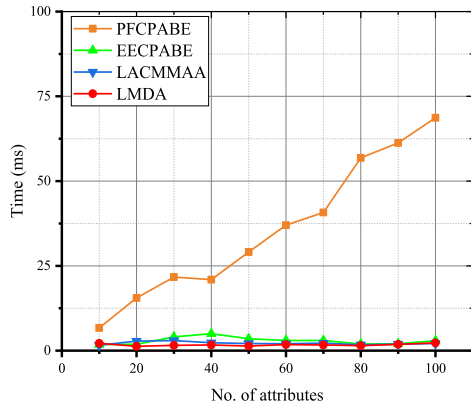


Fig. 6. Comparison of decryption time overhead.

attribute configurations. The total execution time refers to the sum of the execution times of all algorithms in the scheme.

As shown in the figure, our scheme achieves a significant reduction in overall execution time compared with PFCPABE and EECPABE. Meanwhile, the performance gap between our scheme and LACMMAA is relatively small, and their results are comparatively close.

In summary, the experimental results demonstrate that our scheme achieves a significantly lower computational overhead than the PFCPABE and EECPABE schemes. Meanwhile, its computational overhead is comparable to that of the LACMMAA scheme.

#### E. Revocation and Verification Overhead

Attribute revocation involves two smart contracts: the revocation list upload contract and the revocation list query contract. Fig. 8 shows the average latency of both contracts

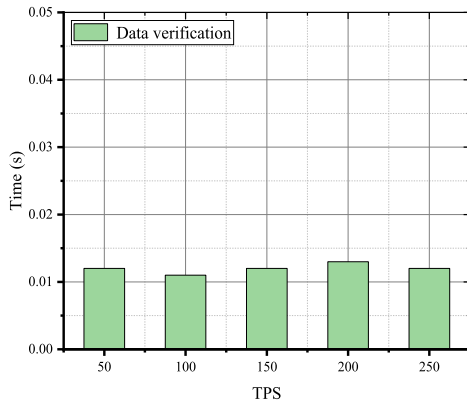


Fig. 9. Average latency of data verification.

under different TPS settings. Under the Raft consensus algorithm, the revocation list upload contract exhibits the lowest average latency at 100 TPS. Although the average latency gradually increases once the sending rate exceeds 100 TPS, it remains below 0.2 s even at 250 TPS. The average latency of the revocation list query contract remains around 0.01 s at the same sending-rate interval. This result indicates that the contract can still maintain relatively stable performance under a large number of concurrent requests and high load. It can also respond quickly enough to meet the requirements of SWIM attribute revocation.

The data trust verification stage involves a single smart contract that validates data integrity. Fig. 9 presents the average latency of this data verification contract under different TPS settings. Within the same sending-rate interval as the revocation list upload contract, the average latency consistently remains around 0.01 s, which falls within a low-latency range. This result indicates that the contract maintains robust performance under high-load conditions and satisfies the real-time requirements of SWIM data verification.

## VIII. CONCLUSION

In this article, we propose a lightweight multiauthority CP-ABE scheme based on dynamic attributes tailored for resource-constrained SWIM environments. By replacing bilinear pairings with elliptic-curve scalar multiplication and outsourcing decryption to semitrusted nodes, the scheme substantially reduces client-side computational overhead. Dynamic attributes derived from SWIM information exchange models further enhance fine-grained access control, while the integration of blockchain enables trustworthy public key verification and efficient attribute revocation. Security analysis shows that our scheme achieves IND-CCA security under the DDH assumption and is resistant to collusion attacks. Experimental evaluations confirm its advantages in terms of computational cost and communication overhead over existing schemes. These results demonstrate that our scheme is well suited for real-world SWIM and aviation IoT applications. Future work will focus on further optimizing the revocation mechanism to reduce the overhead of frequent attribute updates in highly dynamic SWIM scenarios, while preserving fine-grained access control.

## REFERENCES

- [1] Z. Wu, S. Zhou, L. Liu, and J. Lei, "Research on SWIM services dynamic migration method," *Future Internet*, vol. 11, no. 9, p. 187, Aug. 2019, doi: [10.3390/fi11090187](https://doi.org/10.3390/fi11090187).
- [2] X. Lu and T. Koga, "System wide information management for heterogeneous information sharing and interoperability," in *Proc. IEEE 13th Int. Symp. Auto. Decentralized Syst. (ISADS)*, Mar. 2017, pp. 199–204, doi: [10.1109/ISADS.2017.13](https://doi.org/10.1109/ISADS.2017.13).
- [3] G. Li and Z. Wu, "Information-centric networking cache robustness strategy for system wide information management," *IEEE Access*, vol. 8, pp. 82432–82441, 2020, doi: [10.1109/ACCESS.2020.2991762](https://doi.org/10.1109/ACCESS.2020.2991762).
- [4] M. Moallemi, C. A. Castro-Pena, M. Towhidnejad, and B. Abraham, "Information security in the aircraft access to system wide information management infrastructure," in *Proc. Integr. Commun. Navig. Surveill. (ICNS)*, Herndon, VA, USA, Apr. 2016, pp. 1A3-1–1A3-7, doi: [10.1109/ICNSURV.2016.7486321](https://doi.org/10.1109/ICNSURV.2016.7486321).
- [5] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2007, pp. 321–334, doi: [10.1109/SP.2007.11](https://doi.org/10.1109/SP.2007.11).
- [6] Q. Zhang, L. Yuan, T. Xie, and H. Chen, "Audit and dynamic access control scheme with behavior and identity tracing," *Comput. Netw.*, vol. 251, Sep. 2024, Art. no. 110623, doi: [10.1016/j.comnet.2024.110623](https://doi.org/10.1016/j.comnet.2024.110623).
- [7] Y. Ren, C. Chen, M. Hu, G. Feng, and X. Zhang, "BFDAC: A blockchain-based and fog-computing-assisted data access control scheme in vehicular social networks," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 3510–3523, Jan. 2024, doi: [10.1109/JIOT.2023.3296906](https://doi.org/10.1109/JIOT.2023.3296906).
- [8] M. Xiao, H. Li, Q. Huang, S. Yu, and W. Susilo, "Attribute-based hierarchical access control with extendable policy," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1868–1883, 2022, doi: [10.1109/TIFS.2022.3173412](https://doi.org/10.1109/TIFS.2022.3173412).
- [9] Y. Miao et al., "Verifiable outsourced attribute-based encryption scheme for cloud-assisted mobile e-health system," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 1845–1862, Jul. 2024, doi: [10.1109/TDSC.2023.3292129](https://doi.org/10.1109/TDSC.2023.3292129).
- [10] S. Tu, M. Waqas, F. Huang, G. Abbas, and Z. H. Abbas, "A revocable and outsourced multi-authority attribute-based encryption scheme in fog computing," *Comput. Netw.*, vol. 195, Aug. 2021, Art. no. 108196, doi: [10.1016/j.comnet.2021.108196](https://doi.org/10.1016/j.comnet.2021.108196).
- [11] D. Han, N. Pan, and K.-C. Li, "A traceable and revocable ciphertext-policy attribute-based encryption scheme based on privacy protection," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 316–327, Jan. 2022, doi: [10.1109/TDSC.2020.2977646](https://doi.org/10.1109/TDSC.2020.2977646).
- [12] F. Meng and L. Cheng, "TSR-ABE: Traceable and server-aided revocable ciphertext-policy attribute-based encryption under static assumptions," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 955–967, 2025, doi: [10.1109/TIFS.2024.3516542](https://doi.org/10.1109/TIFS.2024.3516542).
- [13] M. Yang, H. Wang, and D. He, "PM-ABE: Puncturable bilateral fine-grained access control from lattices for secret sharing," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 1210–1223, Mar. 2025, doi: [10.1109/TDSC.2024.3431035](https://doi.org/10.1109/TDSC.2024.3431035).
- [14] H. Zhong, W. Zhu, Y. Xu, and J. Cui, "Multi-authority attribute-based encryption access control scheme with policy hidden for cloud storage," *Soft Comput.*, vol. 22, no. 1, pp. 243–251, Jan. 2018, doi: [10.1007/s00500-016-2330-8](https://doi.org/10.1007/s00500-016-2330-8).
- [15] Z. Zhang, J. Zhang, Y. Yuan, and Z. Li, "An expressive fully policy-hidden ciphertext policy attribute-based encryption scheme with credible verification based on blockchain," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8681–8692, Jun. 2022, doi: [10.1109/JIOT.2021.3117378](https://doi.org/10.1109/JIOT.2021.3117378).
- [16] S. Ding, C. Li, and H. Li, "A novel efficient pairing-free CP-ABE based on elliptic curve cryptography for IoT," *IEEE Access*, vol. 6, pp. 27336–27345, 2018, doi: [10.1109/ACCESS.2018.2836350](https://doi.org/10.1109/ACCESS.2018.2836350).
- [17] S. Ding, J. Cao, C. Li, K. Fan, and H. Li, "A novel attribute-based access control scheme using blockchain for IoT," *IEEE Access*, vol. 7, pp. 38431–38441, 2019, doi: [10.1109/ACCESS.2019.2905846](https://doi.org/10.1109/ACCESS.2019.2905846).
- [18] K. Sowjanya, M. Dasgupta, and S. Ray, "A lightweight key management scheme for key-escrow-free ECC-based CP-ABE for IoT healthcare systems," *J. Syst. Archit.*, vol. 117, Aug. 2021, Art. no. 102108, doi: [10.1016/j.sysarc.2021.102108](https://doi.org/10.1016/j.sysarc.2021.102108).
- [19] R. Cheng, K. Wu, Y. Su, W. Li, W. Cui, and J. Tong, "An efficient ECC-based CP-ABE scheme for power IoT," *Processes*, vol. 9, no. 7, p. 1176, Jul. 2021, doi: [10.3390/pr9071176](https://doi.org/10.3390/pr9071176).
- [20] F. Sammy and S. M. C. Vigila, "An efficient blockchain based data access with modified hierarchical attribute access structure with CP-ABE using ECC scheme for patient health record," *Secur. Commun. Netw.*, vol. 2022, no. 1, Mar. 2022, Art. no. 8685273, doi: [10.1155/2022/8685273](https://doi.org/10.1155/2022/8685273).

- [21] S. Das and S. Namasudra, "Multiauthority CP-ABE-based access control model for IoT-enabled healthcare infrastructure," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 821–829, Jan. 2023, doi: [10.1109/TII.2022.3167842](https://doi.org/10.1109/TII.2022.3167842).
- [22] B. Xie, Y.-P. Zhou, X.-Y. Yi, and C.-Y. Wang, "An improved multi-authority attribute access control scheme base on blockchain and elliptic curve for efficient and secure data sharing," *Electronics*, vol. 12, no. 7, p. 1691, Apr. 2023, doi: [10.3390/electronics12071691](https://doi.org/10.3390/electronics12071691).
- [23] S. Das and S. Namasudra, "MACPABE: Multi-Authority-based CP-ABE with efficient attribute revocation for IoT-enabled healthcare infrastructure," *Int. J. Netw. Manage.*, vol. 33, no. 3, p. 2200, May 2023, doi: [10.1002/nem.2200](https://doi.org/10.1002/nem.2200).
- [24] Z. Wu, Z. Cui, C. Wang, and J. Lei, "Access control scheme with attribute revocation for SWIM," *J. China Universities Posts Telecommun.*, vol. 24, no. 6, pp. 49–54, Dec. 2017, doi: [10.1016/s1005-8885\(17\)60241-3](https://doi.org/10.1016/s1005-8885(17)60241-3).
- [25] Z. Wu, J. Nie, Y. Yin, and H. Wang, "Verified CSAC-based CP-ABE access control of cloud storage in SWIM," in *Proc. IEEE 19th Int. Conf. Embedded Ubiquitous Comput. (EUC)*, Oct. 2021, pp. 72–78, doi: [10.1109/EUC53437.2021.00019](https://doi.org/10.1109/EUC53437.2021.00019).
- [26] L. Zhang, Z. You, K. Wang, and Z. Cui, "Research on access control scheme of system wide information management based on attribute association," *Secur. Commun. Netw.*, vol. 2022, no. 1, May 2022, Art. no. 6181995, doi: [10.1155/2022/6181995](https://doi.org/10.1155/2022/6181995).
- [27] Q. Wang, L. Zhang, X. Lu, and K. Wang, "A multi-authority CP-ABE scheme based on cloud-chain fusion for SWIM," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Social Comput. Netw. (ISPA/BDCloud/SocialCom/SustainCom)*, Dec. 2022, pp. 213–219, doi: [10.1109/ISPA-BDCloud-SocialCom-SustainCom57177.2022.00034](https://doi.org/10.1109/ISPA-BDCloud-SocialCom-SustainCom57177.2022.00034).
- [28] X. Lu, Z. Wu, and J. Cao, "ATMChain: Blockchain-based security architecture for air traffic management in future," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 4, pp. 3872–3896, Aug. 2024, doi: [10.1109/TAES.2024.3371396](https://doi.org/10.1109/TAES.2024.3371396).
- [29] V. S. Miller, "Use of elliptic curves in cryptography," in *Advances in Cryptology-(CRYPTO)*, vol. 218, H. C. Williams, Ed., Berlin, Germany: Springer, 1986, pp. 417–426, doi: [10.1007/3-540-39799-X\\_31](https://doi.org/10.1007/3-540-39799-X_31).
- [30] N. Koblitz, "Elliptic curve cryptosystems," *Math. Comput.*, vol. 48, no. 177, pp. 203–209, 1987, doi: [10.1090/s0025-5718-1987-0866109-5](https://doi.org/10.1090/s0025-5718-1987-0866109-5).
- [31] X. Qin, Y. Huang, and X. Li, "An ECC-based access control scheme with lightweight decryption and conditional authentication for data sharing in vehicular networks," *Soft Comput.*, vol. 24, no. 24, pp. 18881–18891, Dec. 2020, doi: [10.1007/s00500-020-05117-x](https://doi.org/10.1007/s00500-020-05117-x).
- [32] A. Lewko, T. Okamoto, A. Sahai, K. Takashima, and B. Waters, "Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption," in *Advances in Cryptology-(EUROCRYPT)*, vol. 6110, H. Gilbert, Ed., Berlin, Germany: Springer, 2010, pp. 62–91, doi: [10.1007/978-3-642-13190-5\\_4](https://doi.org/10.1007/978-3-642-13190-5_4).
- [33] Z. Li and G. Shi, "A CCA-secure puncturable attribute-based proxy re-encryption scheme," *IEEE Internet Things J.*, vol. 12, no. 22, pp. 47679–47690, Nov. 2025, doi: [10.1109/JIOT.2025.3604341](https://doi.org/10.1109/JIOT.2025.3604341).
- [34] W. Luo, Z. Lv, L. Yang, G. Han, and X. Zhang, "FOC-PH-CP-ABE: An efficient CP-ABE scheme with fully outsourced computation and policy hidden in the industrial Internet of Things," *IEEE Sensors J.*, vol. 24, no. 18, pp. 28971–28981, Sep. 2024, doi: [10.1109/JSEN.2024.3432276](https://doi.org/10.1109/JSEN.2024.3432276).



**Yiao Ma** received the B.S. degree in computer science and technology from the Civil Aviation University of China, Tianjin, China, in 2023, where he is currently pursuing the M.S. degree with the School of Safety Science and Engineering.

His research interests include access control and privacy protection of large-scale information.



**Zhijun Wu** received the Ph.D. degree in cryptography from Beijing University of Posts and Telecommunications, Beijing, China, in 2004.

He is currently a Professor with the School of Safety Science and Engineering, Civil Aviation University of China, Tianjin, China. His research interests include the security of air traffic management and System Wide Information Management.



**Kenian Wang** received the M.S. degree in electronic science and technology from Xi'an University of Posts and Telecommunications, Xi'an, China, in 2013.

He is currently an Associate Professor with the Institute of Scientific Innovation, Civil Aviation University of China, Tianjin, China. His research interests include information security assessment of the airborne network and verification of the airborne software.



**Lizhe Zhang** received the Ph.D. degree in safety science and engineering from the Civil Aviation University of China, Tianjin, China, in 2024.

He is currently a Lecturer with the School of Safety Science and Engineering, Civil Aviation University of China. His research interests include the security of air traffic management and system wide information management.



**Ruiqi Li** received the Ph.D. degree in computer science and technology from Nankai University, Tianjin, China, in 2021.

He is currently a Lecturer with the School of Safety Science and Engineering, Civil Aviation University of China, Tianjin. His research interests include fully homomorphic encryption, applied cryptography, and cloud computing security.